

Software Requirements Specification (SRS)

Simple Shell (Command Line Interpreter)

Project: Simple Shell (Command Line Interpreter)

Version: 1.0

Status: Draft / For Review

Date: 08-09-2026

Course Instructor: Prof. Pavithra

Authors / Team Members

- Aarushi Singh — PES2UG24CS015
- Akshaya M — PES2UG24CS050
- Akshay Anil Kumar — PES2UG24CS049
- Adarsh Shivanand Madli — PES2UG24CS028

Revision History

Version	Date	Author(s)	Change summary	Approval
1.0	08-09-2026	Aarushi Singh, Akshaya M, Akshay Anil Kumar, Adarsh Shivanand Madli	Initial draft SRS for Simple Shell project	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator	Prof. Pavithra		

Table of Contents

1. Introduction
2. Overall Description
3. External Interface Requirements
4. System Features (Detailed)
5. Non-Functional Requirements (Detailed)
6. Quality Attributes & Acceptance Tests
7. UML Use-Case Diagrams
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document is a Software Requirements Specification (SRS) for the Simple Shell project, a command-line interpreter (CLI) that allows a user to enter, parse, and execute commands on a Linux-based operating system. It defines the functional and non-functional requirements, external interfaces, security requirements, and verification criteria for the system, and is intended for use by the development team, course instructor, and evaluators of this Software Engineering project.

1.2 Scope

The Simple Shell provides a text-based command-line interface through which a user can type commands and have them executed either as built-in shell operations or as external operating-system programs. The shell supports foreground and background execution, input/output redirection, pipes between two or more commands, environment variable expansion, a bounded command history, and basic signal handling (Ctrl+C). The scope of this project is restricted to a single-user, single-session, locally running shell implemented in C on a POSIX-compliant Linux environment. The following are explicitly excluded from scope: shell scripting languages, aliasing, shell functions, job control (fg/bg/jobs), advanced globbing/wildcards beyond simple pattern matching, networking, remote command execution, and user authentication.

1.3 Audience

This document is intended for: the four student developers implementing the shell, the course instructor and teaching assistants evaluating the project, and any peer reviewers assessing the design and test coverage of the system.

1.4 Definitions

- CLI — Command Line Interface
- Shell — A command-line interpreter program that reads and executes user commands
- Built-in command — A command implemented directly inside the shell process (e.g., cd, pwd)
- External command — A program located on the file system and executed as a separate process (e.g., /bin/ls)
- PATH — An environment variable listing directories searched for executable programs
- PID — Process Identifier
- stdin/stdout/stderr — Standard input, standard output, and standard error streams
- Redirection — Changing the source or destination of a command's standard streams using <, >, or >>
- Pipe — A mechanism (|) connecting the standard output of one command to the standard input of another
- SIGINT — The interrupt signal generated by Ctrl+C
- Zombie process — A terminated child process whose exit status has not yet been collected by its parent
- POSIX — Portable Operating System Interface, a family of standards for maintaining compatibility between operating systems

2. Overall Description

2.1 Product Perspective

The Simple Shell is a standalone, self-contained user-mode application; it is not a client-server system and has no dependency on network services. It runs as a single foreground process on top of a POSIX-compliant

Linux kernel, using standard operating-system services (process creation, file I/O, signal delivery) to launch and manage other programs on the user's behalf. The shell replaces (for the duration of its use) the default login shell as the point of interaction between the user and the operating system.

2.2 Major Product Functions

- Display an interactive command prompt and accept typed commands
- Parse command lines into a command name and a list of arguments
- Execute a fixed set of built-in commands (cd, pwd, echo, help, exit, history)
- Locate and execute external programs found via the PATH environment variable
- Run commands in the foreground (blocking) or background (using &)
- Perform basic process management, including reaping of completed background processes
- Redirect standard input and standard output using <, >, and >>
- Connect two or more commands using pipes (|)
- Expand environment variables referenced in command arguments
- Maintain and display a bounded history of previously entered commands
- Handle the Ctrl+C (SIGINT) signal without terminating the shell itself
- Report clear, specific error messages for invalid commands, missing files, or malformed input

2.3 User Roles and Characteristics

- Student / Developer: Uses the shell to compile and run programs during coursework; comfortable with basic Linux commands.
- System Administrator (representative role): Uses the shell for routine file and process management tasks; expects predictable, POSIX-like behaviour.
- Course Evaluator: Reviews the shell's behaviour against the requirements and test cases defined in this document; does not require prior familiarity with the implementation.

2.4 Operating Environment

The shell is implemented in C and targets a POSIX-compliant Linux distribution (e.g., Ubuntu 20.04 LTS or later) with the GNU C library (glibc), compiled using GCC. It requires a standard terminal emulator for input and output and does not require any dedicated hardware beyond a standard keyboard and display.

2.5 Constraints

- The system shall be implemented in the C programming language using standard POSIX system calls (fork, execve/execvp, wait/waitpid, pipe, dup2, open, chdir, getcwd, signal/sigaction).
- The shell provides a text-based interface only; no graphical user interface is in scope.
- The shell supports a single interactive user per running instance; multi-user/remote access is out of scope.
- Command history and configuration are held in process memory and, where persisted, in a plain local file; no database or external storage service is used.

2.6 Assumptions and Dependencies

- The host system provides a POSIX-compliant kernel and a standard C runtime library.
- The user has a properly configured PATH environment variable for locating external commands.
- The underlying file system correctly enforces file and directory permissions; the shell relies on, rather than re-implements, these OS-level protections.

3. External Interface Requirements

3.1 User Interfaces

The primary user interface is a text-based command-line interface presented in a terminal emulator. The shell displays a prompt (e.g., `shell>`) indicating it is ready to accept input, echoes user keystrokes via the terminal driver, and prints command output, error messages, and history listings as plain text to standard output/standard error. No graphical widgets, menus, or mouse interaction are provided.

3.2 Hardware Interfaces

The shell requires only a standard keyboard for input and a standard text-capable display (terminal) for output, both mediated by the operating system's terminal driver (tty). No specialised or dedicated hardware devices (card readers, printers, sensors, etc.) are used or required.

3.3 Software Interfaces

- Operating System APIs / System Calls: `fork()`, `execvp()`, `wait()/waitpid()`, `pipe()`, `dup2()`, `open()/close()`, `chdir()`, `getcwd()`, `signal()/sigaction()`, `getenv()/setenv()`.
- C Standard Library: for string handling, memory management, and I/O buffering (e.g., `stdio.h`, `string.h`, `stdlib.h`).
- Underlying File System: for resolving relative/absolute paths, redirection targets, and the `PATH` search.

3.4 Communications Interfaces

The Simple Shell does not communicate over a network. Inter-process communication is limited to local, in-kernel mechanisms — specifically anonymous pipes (`pipe()`) used to connect the standard output of one child process to the standard input of another when the pipe operator (`|`) is used. No remote procedure calls, sockets, or external service integrations are part of this system.

4. System Features (Detailed)

Each functional requirement below includes its acceptance criteria and a reference test case. Requirement IDs follow the pattern `SH-F-####`. Requirements are grouped into modules that correspond to the intended software architecture of the system.

4.1 Command Input and Parsing

Module: Command Input Module (CIM), Command Parser (CP). Description: read a line of user input, tokenize it into a command and its arguments, and prepare it for execution.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SH-F-001	The system shall display a command prompt (e.g., <code>shell></code>) and wait for user input.	Functional	High	End User	AC-SH-F-001: On shell start-up and after every command completes, a prompt is visible and the shell accepts keyboard input. Test: TC-CLI-01	None
SH-F-002	The system shall accept command input up to a maximum configured command-line length and shall safely reject input exceeding this limit without crashing.	Functional	High	End User	AC-SH-F-002: Input within the configured limit is read correctly; input exceeding the limit is safely rejected with a clear message and the shell continues running without crashing. Test: TC-CLI-02	Depends on CIM buffer size
SH-F-003	The system shall tokenize the input line into a command	Functional	High	Developer	AC-SH-F-003: A line such as <code>'ls -l /tmp'</code> is parsed into	Depends on CIM

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	name and zero or more arguments, splitting on whitespace.				command 'ls' with arguments ['-l', '/tmp']. Test: TC-PARSE-01	
SH-F-004	The system shall support single- and double-quoted arguments, treating whitespace within matching quotes as part of a single argument.	Functional	Medium	Developer	AC-SH-F-004: The input echo "hello world" is parsed as a single argument 'hello world'. Test: TC-PARSE-02	Depends on CP
SH-F-005	The system shall ignore blank or whitespace-only input lines and redisplay the prompt without attempting execution.	Functional	Medium	End User	AC-SH-F-005: Pressing Enter on an empty line redisplay the prompt with no error. Test: TC-PARSE-03	None

4.2 Built-in Commands

Module: Built-in Command Module (BCM). Description: commands implemented directly within the shell process rather than as separate executables.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SH-F-006	The system shall implement the 'cd' built-in command to change the shell's current working directory, supporting an absolute path, a relative path, and no argument (defaulting to the HOME environment variable).	Functional	High	End User	AC-SH-F-006: After 'cd <dir>', a subsequent 'pwd' reflects <dir>; 'cd' with no argument moves to \$HOME. Test: TC-BLTIN-CD-01	Depends on EVM for HOME lookup
SH-F-007	The system shall implement the 'pwd' built-in command to print the absolute path of the current working directory.	Functional	High	End User	AC-SH-F-007: 'pwd' output matches the OS-reported current working directory. Test: TC-BLTIN-PWD-01	None
SH-F-008	The system shall implement the 'echo' built-in command to print its arguments, separated by single spaces, followed by a newline.	Functional	High	End User	AC-SH-F-008: 'echo hello world' prints 'hello world' followed by a newline. Test: TC-BLTIN-ECHO-01	Depends on EVM for variable expansion
SH-F-009	The system shall implement a 'help' built-in command that lists all supported built-in commands with a one-line description of each.	Functional	Medium	End User	AC-SH-F-009: Running 'help' lists cd, pwd, echo, help, exit, and history with descriptions. Test: TC-BLTIN-HELP-01	None
SH-F-010	The system shall implement an 'exit' built-in command that terminates the shell process cleanly, releasing any allocated resources.	Functional	High	End User	AC-SH-F-010: Running 'exit' terminates the shell with exit status 0 and returns control to the parent process/terminal. Test: TC-BLTIN-EXIT-01	None
SH-F-011	The system shall implement a 'history' built-in command that prints the list of previously entered commands, numbered in order of entry.	Functional	Medium	End User	AC-SH-F-011: After entering 3 commands, 'history' lists all 3 in order with sequence numbers. Test: TC-HIST-02	Depends on History Module

4.3 External Command Execution

Module: Process Execution Module (PEM). Description: locate and run programs that are not built-in commands.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SH-F-012	The system shall search the directories listed in the PATH environment variable, in order, to locate the executable file for a non-built-in command.	Functional	High	Developer	AC-SH-F-012: Entering 'ls' resolves to /bin/ls (or equivalent) using PATH, without the user specifying the full path. Test: TC-EXEC-01	Depends on EVM
SH-F-013	The system shall execute the located external program as a child process using fork() and execvp(), passing the parsed arguments.	Functional	High	Developer	AC-SH-F-013: A child process is created and the target program's output appears on the terminal. Test: TC-EXEC-02	Depends on PEM
SH-F-014	The system shall wait for a foreground child process to complete before redisplaying the prompt, unless the command was launched in the background.	Functional	High	End User	AC-SH-F-014: For a foreground command, the prompt does not reappear until the child process exits. Test: TC-EXEC-02	Depends on PEM
SH-F-015	The system shall display a clear error message (e.g., 'command not found') and return to the prompt if the command cannot be located or executed.	Functional	High	End User	AC-SH-F-015: Entering a non-existent command name prints 'command not found' and the prompt reappears without crashing. Test: TC-EXEC-03	Depends on Error Handling Module

4.4 Process Management

Module: Process Execution Module (PEM). Description: management of foreground and background child processes.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SH-F-016	The system shall support launching a command in the background when the input line ends with the '&' character, returning control to the prompt immediately.	Functional	High	End User	AC-SH-F-016: 'sleep 5 &' returns to the prompt immediately without waiting 5 seconds. Test: TC-PROC-01	Depends on PEM
SH-F-017	The system shall display the process ID (PID) of a newly launched background job to the user.	Functional	Medium	End User	AC-SH-F-017: Launching a background job prints a message containing its PID. Test: TC-PROC-02	Depends on PEM
SH-F-018	The system shall periodically reap completed background child processes to prevent the accumulation of zombie processes.	Functional	High	Developer	AC-SH-F-018: After a background job completes, its entry does not appear as a zombie ('defunct') process in the process table. Test: TC-PROC-03	Depends on PEM, Signal Handling Module

4.5 Input/Output Redirection

Module: I/O Redirection Module (IRM). Description: redirect a command's standard input and/or standard output to files.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SH-F-019	The system shall support input redirection using '<',	Functional	High	Developer	AC-SH-F-019: 'sort < input.txt' reads its input	Depends on IRM

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	causing the specified command to read its standard input from the named file instead of the keyboard.				from input.txt rather than the terminal. Test: TC-REDIR-01	
SH-F-020	The system shall support output redirection using '>', creating or truncating the named file and writing the command's standard output to it.	Functional	High	Developer	AC-SH-F-020: 'ls > out.txt' creates/overwrites out.txt with the directory listing instead of printing to the terminal. Test: TC-REDIR-02	Depends on IRM
SH-F-021	The system shall support output-append redirection using '>>', creating the named file if it does not exist and appending the command's standard output to it.	Functional	Medium	Developer	AC-SH-F-021: Running 'echo hi >> log.txt' twice results in log.txt containing 'hi' on two separate lines. Test: TC-REDIR-03	Depends on IRM
SH-F-022	The system shall report a clear error message if a redirection target file cannot be opened (e.g., due to missing permissions or a non-existent directory).	Functional	Medium	End User	AC-SH-F-022: Redirecting output to a path inside a non-writable directory produces an error message and no crash. Test: TC-REDIR-02	Depends on IRM, Error Handling Module

4.6 Pipes

Module: Pipe Management Module (PMM). Description: connect the output of one command to the input of the next.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SH-F-023	The system shall support connecting two commands with a single pipe (' '), directing the standard output of the first command into the standard input of the second.	Functional	High	Developer	AC-SH-F-023: 'ls wc -l' prints the count of entries in the current directory. Test: TC-PIPE-01	Depends on PMM
SH-F-024	The system shall support chaining more than two commands with multiple pipe operators on a single input line.	Functional	Medium	Developer	AC-SH-F-024: 'ls grep .c wc -l' correctly reports the count of matching files. Test: TC-PIPE-02	Depends on PMM

4.7 Environment Variables

Module: Environment Variable Module (EVM). Description: expand and expose shell/OS environment variables to commands.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SH-F-025	The system shall expand references to environment variables (e.g., \$HOME, \$PATH, \$USER) in command arguments before execution.	Functional	Medium	Developer	AC-SH-F-025: 'echo \$HOME' prints the value of the HOME environment variable. Test: TC-ENV-01	Depends on EVM
SH-F-026	The system shall pass the current process environment to any external command it launches.	Functional	High	Developer	AC-SH-F-026: An external program invoked from the shell can read environment variables such as PATH and HOME. Test: TC-EXEC-02	Depends on EVM, PEM

4.8 Command History

Module: History Module (HM). Description: record a bounded list of previously entered commands.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SH-F-027	The system shall maintain an in-memory history of the most recently entered commands, up to a configurable limit of at least 50 entries.	Functional	Medium	End User	AC-SH-F-027: After entering more than 50 commands, the history contains only the most recent 50, in order. Test: TC-HIST-01	Depends on HM
SH-F-028	The system shall exclude blank input lines from the recorded command history.	Functional	Low	End User	AC-SH-F-028: Pressing Enter on an empty line does not add an entry to the history list. Test: TC-HIST-01	Depends on HM, CIM

4.9 Signal Handling and Error Management

Module: Signal Handling Module (SHM), Error Handling Module (EHM). Description: keep the shell responsive and stable in the presence of interrupts and invalid input.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SH-F-029	The system shall intercept the SIGINT signal (Ctrl+C) so that it terminates only the currently running foreground child process, if any, without terminating the shell itself.	Functional	High	End User	AC-SH-F-029: Pressing Ctrl+C while a foreground command such as 'sleep 30' is running terminates that command and returns to the prompt; the shell keeps running. Test: TC-SIG-01	Depends on SHM
SH-F-030	The system shall print a specific, human-readable error message for each recognised failure category (command not found, permission denied, file not found, invalid syntax).	Functional	High	End User	AC-SH-F-030: Each failure category produces a distinct, descriptive message rather than a generic failure code. Test: TC-ERR-01	Depends on EHM
SH-F-031	The system shall detect malformed input, such as unmatched quotes or a pipe/redirection operator with a missing operand, report an error, and continue running without crashing.	Functional	High	Developer	AC-SH-F-031: Input such as 'echo "unterminated' or a trailing ' ' produces a syntax error message and the shell remains usable. Test: TC-ERR-02	Depends on CP, EHM

5. Non-Functional Requirements (Detailed)

The non-functional requirements below are measurable and are each tied to a verification method. Requirement IDs follow the pattern SH-NF-####.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
SH-NF-001	The shell shall display the next prompt within 1 second after completion of a foreground command under normal system load.	Performance	High	Measured over 20 consecutive command executions on the target test machine under normal load; prompt reappears within 1 second in each case. Test: TC-PERF-01

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
SH-NF-002	The shell shall not terminate or crash (e.g., segmentation fault) when subjected to the full functional and negative test suite defined in Section 6.	Reliability	High	Zero crashes across all defined test cases in a full test run. Test: TC-REL-01
SH-NF-003	All error and help messages produced by the shell shall be written in plain English, state the nature of the problem, and avoid raw system error codes without explanation.	Usability	Medium	Manual review of all error/help strings against a checklist; 100% pass. Test: TC-USE-01
SH-NF-004	The shell's source code shall be organised into the modules identified in Section 4 (CIM, CP, BCM, PEM, IRM, PMM, EVM, HM, SHM, EHM), each in a separate source file with a documented public interface.	Maintainability	Medium	Code review confirms one module per listed responsibility with header-documented functions. Test: TC-MAINT-01
SH-NF-005	The shell shall compile and execute successfully on at least two supported POSIX-compliant Linux environments.	Portability	Medium	Successful build and smoke test on each of the two target POSIX-compliant Linux environments. Test: TC-PORT-01
SH-NF-006	The shell shall properly reap completed background child processes so that terminated child processes do not accumulate as zombie processes during normal operation.	Resource Efficiency	Medium	Process table inspection during a background-job stress test shows no accumulation of zombie processes during normal operation. Test: TC-RES-01

5.1 Security

5.1.1 Security Objectives

This section lists the security objectives applicable to the Simple Shell, scoped to a local, single-user, non-networked command-line tool.

- SO-1: Prevent the shell from executing unintended, unparsed, or maliciously crafted input as a command, beyond what the user explicitly typed.
- SO-2: Prevent unintended disclosure of environment information or internal system state through error messages, debugging output, or automatic logging, while allowing the user to explicitly request supported environment variables.

5.1.2 Security Requirements

Security requirement IDs follow the pattern SH-SR-###.

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
SH-SR-001	The system shall bound all input buffers (command line and argument storage) to a fixed maximum size and reject or safely truncate input that exceeds this size, preventing buffer overflow.	Security	High	AC-SH-SR-001: Input exceeding the configured command-line limit is rejected or safely truncated without memory corruption or crash. Test: TC-SEC-01
SH-SR-002	The system shall treat command-line input as literal command and argument text only; it shall not pass user input to a shell-evaluation function (e.g., <code>system()</code>) that could allow unintended command chaining or injection.	Security	High	AC-SH-SR-002: Special characters (e.g., <code>;</code> , <code>'</code> , <code>\$()</code>) typed as part of an argument are treated as literal text, not executed, unless explicitly supported (e.g., <code>\$</code> for variable expansion). Test: TC-SEC-02

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
SH-SR-003	The system shall rely on, and never bypass, the underlying operating system's file-permission checks when opening files for redirection or executing programs.	Security	High	AC-SH-SR-003: Attempting to redirect output to a file the user lacks write permission for fails with a permission-denied error, not a silent bypass. Test: TC-SEC-03
SH-SR-004	The system shall not print the full contents of the process environment (e.g., via debug output or unhandled error messages) unless the user explicitly requests a specific variable's value.	Security	Medium	AC-SH-SR-004: Error messages and logs reference variable names where relevant but do not dump the entire environment block. Test: TC-SEC-04
SH-SR-005	The system shall validate redirection and pipe syntax before opening any file or spawning any process, rejecting unsupported or malformed pipe/redirection operator sequences (e.g., consecutive pipe operators or a pipe/redirection operator without a valid operand) with an error rather than attempting a partial, unsafe operation.	Security	Medium	AC-SH-SR-005: Unsupported or malformed redirection/pipe syntax is rejected before any file descriptor is opened or process is forked. Test: TC-SEC-05
SH-SR-006	The system shall ensure that input containing embedded null bytes or other non-printable control characters (other than newline) does not cause undefined behaviour, truncating or rejecting such input safely.	Security	Medium	AC-SH-SR-006: Input containing embedded control characters is handled without crashing, memory corruption, or misparsing beyond the point of the offending byte. Test: TC-SEC-06

6. Quality Attributes & Acceptance Tests

6.1 Acceptance Criteria

- All High-priority functional requirements (SH-F-###) are implemented and verified.
- All defined test cases for High-priority requirements pass; no critical (crash-level) failures remain open.
- All Non-Functional Requirement (SH-NF-###) targets are met as measured by their associated test cases.
- All Security Requirements (SH-SR-###) are implemented and verified against their acceptance criteria.
- The Requirements Traceability Matrix (Section 8) is complete, with every requirement mapped to at least one test case.

6.2 Acceptance Test Suites

- Command Execution Suite: TC-EXEC-01 to TC-EXEC-03 — covers PATH lookup, foreground execution, and command-not-found handling.
- Built-in Commands Suite: TC-BLTIN-CD-01, TC-BLTIN-PWD-01, TC-BLTIN-ECHO-01, TC-BLTIN-HELP-01, TC-BLTIN-EXIT-01 — covers each built-in command.
- Parsing Suite: TC-PARSE-01 to TC-PARSE-03 — covers tokenizing, quoted arguments, and blank-line handling.
- Redirection Suite: TC-REDIR-01 to TC-REDIR-03 — covers input, output, and append redirection.
- Pipe Suite: TC-PIPE-01, TC-PIPE-02 — covers single and multi-command pipelines.

- Process Management Suite: TC-PROC-01 to TC-PROC-03 — covers background execution, PID reporting, and zombie reaping.
- Environment & History Suite: TC-ENV-01, TC-HIST-01, TC-HIST-02 — covers variable expansion and command history.
- Error Handling Suite: TC-ERR-01, TC-ERR-02 — covers error messages and malformed-input handling.
- Performance Suite: TC-PERF-01 — covers prompt response latency.
- Reliability Suite: TC-REL-01 — covers stability across the full negative/functional test set.
- Security Suite: TC-SEC-01 to TC-SEC-06 — covers input bounding, injection resistance, permission handling, information disclosure, syntax validation, and control-character handling.
- Usability / Portability / Maintainability / Resource Suite: TC-USE-01, TC-PORT-01, TC-MAINT-01, TC-RES-01.

7. UML Use-Case Diagrams

Two use-case diagrams are defined below, each given as a PlantUML specification that can be rendered into a graphical diagram (e.g., using a PlantUML renderer or plugin), together with a textual description of the actors, use cases, and relationships.

7.1 Diagram 1 — Basic Command Input and Built-in Commands

Actors: User.

Use Cases: Enter Command, Parse Command, Execute Built-in Command (cd, pwd, echo, help, exit, history), Display Prompt, Display Error Message.

Relationships: 'Enter Command' <<include>> 'Parse Command'. 'Display Error Message' <<extend>> 'Parse Command', triggered when the input is malformed. 'Execute Built-in Command' is triggered after successful parsing when the command matches a built-in.

```
@startuml
actor User
rectangle "Simple Shell" {
    usecase "Enter Command" as UC1
    usecase "Display Prompt" as UC2
    usecase "Parse Command" as UC3
    usecase "Execute Built-in Command" as UC4
    usecase "Display Error Message" as UC5
}
User --> UC2
User --> UC1
UC1 .-> UC3 : <<include>>
UC3 .-> UC4
UC5 .-> UC3 : <<extend>>
@enduml
```

7.2 Diagram 2 — Process Management, Redirection, Pipes, and Environment Handling

Actors: User.

Use Cases: Execute External Command, Run In Background, Redirect Input/Output, Chain Commands With Pipe, Expand Environment Variable, Handle Signal (Ctrl+C), View Command History.

Relationships: 'Execute External Command' <<include>> 'Expand Environment Variable'. 'Run In Background' <<extend>> 'Execute External Command'. 'Chain Commands With Pipe' <<include>> 'Execute External Command' for each stage of the pipeline. 'Handle Signal (Ctrl+C)' <<extend>> 'Execute External Command' while a foreground process is running.

```
@startuml
actor User
rectangle "Simple Shell" {
```

```

usecase "Execute External Command" as UC6
usecase "Run In Background" as UC7
usecase "Redirect Input/Output" as UC8
usecase "Chain Commands With Pipe" as UC9
usecase "Expand Environment Variable" as UC10
usecase "Handle Signal (Ctrl+C)" as UC11
usecase "View Command History" as UC12
}
User --> UC6
User --> UC8
User --> UC9
User --> UC12
UC6 .> UC10 : <<include>>
UC7 .> UC6 : <<extend>>
UC9 .> UC6 : <<include>>
UC11 .> UC6 : <<extend>>
@enduml

```

8. Requirements Traceability Matrix (RTM)

Status codes: N = Not implemented / Not verified, P = Partially implemented / verified, A = Accepted (implemented and verified). All statuses below are initialised to 'N' as this document represents the initial draft SRS; no implementation or test results are claimed at this stage.

Req ID	Requirement (short)	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
SH-F-001	Display prompt	4.1	CIM	TC-CLI-01	N	
SH-F-002	Read input line	4.1	CIM	TC-CLI-02	N	
SH-F-003	Tokenize command line	4.1	CP	TC-PARSE-01	N	
SH-F-004	Support quoted arguments	4.1	CP	TC-PARSE-02	N	
SH-F-005	Ignore blank input	4.1	CIM	TC-PARSE-03	N	
SH-F-006	'cd' built-in	4.2	BCM	TC-BLTIN-CD-01	N	
SH-F-007	'pwd' built-in	4.2	BCM	TC-BLTIN-PWD-01	N	
SH-F-008	'echo' built-in	4.2	BCM	TC-BLTIN-ECHO-01	N	
SH-F-009	'help' built-in	4.2	BCM	TC-BLTIN-HELP-01	N	
SH-F-010	'exit' built-in	4.2	BCM	TC-BLTIN-EXIT-01	N	
SH-F-011	'history' built-in	4.2 / 4.8	BCM, HM	TC-HIST-02	N	
SH-F-012	PATH lookup for external commands	4.3	PEM	TC-EXEC-01	N	
SH-F-013	Fork/exec external command	4.3	PEM	TC-EXEC-02	N	
SH-F-014	Wait for foreground completion	4.3	PEM	TC-EXEC-02	N	
SH-F-015	Command-not-found error	4.3	PEM, EHM	TC-EXEC-03	N	
SH-F-016	Background execution ('&')	4.4	PEM	TC-PROC-01	N	
SH-F-017	Display background PID	4.4	PEM	TC-PROC-02	N	
SH-F-018	Reap zombie processes	4.4	PEM	TC-PROC-03	N	
SH-F-019	Input redirection '<'	4.5	IRM	TC-REDIR-01	N	
SH-F-020	Output redirection '>'	4.5	IRM	TC-REDIR-02	N	
SH-F-021	Append redirection '>>'	4.5	IRM	TC-REDIR-03	N	
SH-F-022	Redirection file-open error	4.5	IRM, EHM	TC-REDIR-02	N	
SH-F-023	Single pipe	4.6	PMM	TC-PIPE-01	N	
SH-F-024	Multi-command pipe chain	4.6	PMM	TC-PIPE-02	N	
SH-F-025	Environment variable expansion	4.7	EVM	TC-ENV-01	N	

Req ID	Requirement (short)	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
SH-F-026	Pass environment to child	4.7	EVM, PEM	TC-EXEC-02	N	
SH-F-027	Maintain bounded history	4.8	HM	TC-HIST-01	N	
SH-F-028	Exclude blank lines from history	4.8	HM, CIM	TC-HIST-01	N	
SH-F-029	SIGINT handling (Ctrl+C)	4.9	SHM	TC-SIG-01	N	
SH-F-030	Specific error messages	4.9	EHM	TC-ERR-01	N	
SH-F-031	Malformed-input handling	4.9	CP, EHM	TC-ERR-02	N	
SH-NF-001	Prompt response time	5	CIM	TC-PERF-01	N	
SH-NF-002	No crashes under test suite	5	All Modules	TC-REL-01	N	
SH-NF-003	Plain-English error messages	5	EHM	TC-USE-01	N	
SH-NF-004	Modular source organisation	5	All Modules	TC-MAINT-01	N	
SH-NF-005	Cross-distribution build	5	Build System	TC-PORT-01	N	
SH-NF-006	Background child-process reaping	5	PEM	TC-RES-01	N	
SH-SR-001	Bounded input buffers	5.1.2	CIM	TC-SEC-01	N	
SH-SR-002	No shell-evaluation of raw input	5.1.2	CP, PEM	TC-SEC-02	N	
SH-SR-003	Respect OS file permissions	5.1.2	IRM, PEM	TC-SEC-03	N	
SH-SR-004	No full-environment disclosure	5.1.2	EVM, EHM	TC-SEC-04	N	
SH-SR-005	Validate redirection/pipe syntax	5.1.2	CP, IRM, PMM	TC-SEC-05	N	
SH-SR-006	Safe handling of control characters	5.1.2	CIM, CP	TC-SEC-06	N	